PYTHON AND AI · EPISODE 08

Naming Variables and Swapping Values

Identifier Rules, PEP 8 Names and the Classic Swap

KEY POINTS

- An **identifier** is a name you give to something in your code, like a variable.
- **Rules** (break one and the code won't run): only **letters, digits and** `_`; **no digit first**; **no keywords** (35 of them, like `True`, `if`, `for`); **case sensitive** (`age`, `Age`, `AGE` are three variables).
- **Conventions** from **PEP 8** (the code still runs without them): names that say what they hold, **all lowercase**, words joined with underscores (**snake case**).
- To **swap** two variables, save one value in a **temporary variable** first, or write `a, b = b, a`.

01 The Four Rules

▶ VIDEO 00:51

1 · CHARACTERS	2 · NO DIGIT FIRST	3 · NO KEYWORDS	4 · CASE SENSITIVE
`a-z` `A-Z` `0-9` `_`	`name6` ✓ `6name` ×	`True` `if` `for`	`age` ≠ `Age` ≠ `AGE`

No spaces, dashes or dollar signs. Only `True` with a capital T is a keyword, so lowercase `true` is a legal name.

02 Test It in PyCharm

▶ VIDEO 02:33

names.py — PYTHON

```python
# Identifier rules: letters, digits and underscores; no digit first; no keywords; case sensitive
true = 1  # lowercase true is not a keyword, so it is a legal name
print(true)

name6 = "Python"  # a digit is fine, just not at the start
print(name6)

age = 1
Age = 2
AGE = 3  # three different variables
print(age, Age, AGE)
```

`True = 1` stops with `SyntaxError: cannot assign to True`. `6name = "Python"` is underlined in red straight away and stops with `SyntaxError: invalid decimal literal`.

03 Conventions from PEP 8

▶ VIDEO 03:20

KEY IDEA

Rules vs conventions: break a rule and the code won't run; skip a convention and it still runs, but it's harder to read. PEP 8 is Python's official style guide (first published in 2001): code is read much more often than it is written.

Use `name`, `age`, `color`, not `a`, `a1`, `a2`. Join words with underscores: `update_time`, `user_first_name`. Write `user_name`, not `UserName`.

04 Swap Two Variables

▶ VIDEO 04:45

`a = b` followed by `b = a` leaves **both** as 20: the 10 is overwritten before it is saved. Like swapping two drinks, you need a third glass.

swap.py — PYTHON

```python
# Swap two variables with a temporary variable
a = 10
b = 20

temp = a  # save the value of a: temp holds 10
a = b     # a is now 20
b = temp  # b gets the original 10

print(a, b)

# The Python shortcut: the right side is worked out first
a, b = b, a
print(a, b)
```

`a, b = b, a` works because Python works out the whole right side first, then stores the values on the left.

05 Try It: Three-Way Swap

▶ VIDEO 07:26

three_swap.py — PYTHON

```python
# Three-way swap: a's value goes into c, b's into a, c's into b
a = 100
b = 200
c = 300

temp = a
a = b
b = c
c = temp
print(a, b, c)

# The same in one line
a, b, c = 100, 200, 300
a, b, c = b, c, a
print(a, b, c)
```

CODE 6 files on GitHub

github.com/abitclearer/python-and-ai/tree/main/08-naming-variables-and-swapping-values